

# Estudo de Viabilidade — Exposição de Dados do SonarQube no Autosserviço

Task #145475 — Estudo de Viabilidade – Exposição de Dados do SonarQube no Autosserviço

Projeto: COTIC - Auto Serviço | Ciclo 010 - Mar/26

## Sumário

1. Endpoints Disponíveis na API
2. Métricas Relevantes para o Autosserviço
3. Autenticação
4. Estrutura de Dados Proposta para o Frontend
5. Arquitetura de Integração
6. Considerações Técnicas
7. Conclusão e Estimativa de Esforço

## 1. Endpoints Disponíveis na API

A API Web do SonarQube expõe endpoints REST que permitem consultar métricas, issues, quality gates e histórico de análises. Os mais relevantes para o Autosserviço:

| Endpoint                             | Finalidade                                                       | Relevância                                     |
|--------------------------------------|------------------------------------------------------------------|------------------------------------------------|
| GET /api/measures/component          | Métricas de um projeto (cobertura, bugs, vulnerabilidades, etc.) | <b>Alta</b> — Endpoint principal               |
| GET /api/qualitygates/project_status | Status do Quality Gate (OK/ERROR) com condições                  | <b>Alta</b> — Semáforo visual                  |
| GET /api/issues/search               | Busca de issues com filtros e facets de agregação                | <b>Alta</b> — Distribuição por severidade/tipo |
| GET /api/measures/search_history     | Histórico temporal de métricas                                   | <b>Média</b> — Gráficos de tendência           |
| GET /api/project_analyses/search     | Histórico de análises com eventos                                | <b>Média</b> — Data da última análise          |
| GET /api/metrics/search              | Lista todas as métricas disponíveis                              | Baixa — Descoberta dinâmica                    |
| GET /api/projects/search             | Lista projetos cadastrados                                       | Baixa — Inventário                             |

### 1.1 GET /api/measures/component

Retorna métricas de um componente (projeto, módulo, diretório ou arquivo).

| Parâmetro        | Obrigatório | Descrição                               |
|------------------|-------------|-----------------------------------------|
| component        | Sim         | Chave do projeto (project key)          |
| metricKeys       | Sim         | Lista de métricas separadas por vírgula |
| additionalFields | Não         | Campos extras: periods , metrics        |
| branch           | Não         | Nome da branch                          |

**Exemplo de requisição:**

GET /api/measures/component?component=meu-projeto&metricKeys=coverage,bugs,vulnerabilities,code\_smells,alert\_stat

#### Exemplo de resposta:

```
{
  "component": {
    "key": "meu-projeto",
    "name": "Meu Projeto",
    "qualifier": "TRK",
    "measures": [
      { "metric": "coverage", "value": "35.6" },
      { "metric": "bugs", "value": "12" },
      { "metric": "vulnerabilities", "value": "3" },
      { "metric": "alert_status", "value": "OK" },
      { "metric": "reliability_rating", "value": "1" }
    ]
  }
}
```

**Nota:** Métricas prefixadas com `new_` retornam dados no campo `periods / period` referente ao “New Code Period”. Ratings são numéricos: 1 = A, 2 = B, 3 = C, 4 = D, 5 = E.

### 1.2 GET /api/qualitygates/project\_status

Retorna o status do Quality Gate com as condições avaliadas.

| Parâmetro  | Obrigatório | Descrição        |
|------------|-------------|------------------|
| projectKey | Sim         | Chave do projeto |
| branch     | Não         | Nome da branch   |

Status possíveis: OK , ERROR , WARN , NONE

### 1.3 GET /api/issues/search

Busca issues (bugs, vulnerabilidades, code smells) com filtros avançados.

| Parâmetro     | Descrição                                           |
|---------------|-----------------------------------------------------|
| componentKeys | Chave(s) do projeto                                 |
| types         | CODE_SMELL , BUG , VULNERABILITY , SECURITY_HOTSPOT |
| severities    | INFO , MINOR , MAJOR , CRITICAL , BLOCKER           |
| facets        | Agregações: severities , types , statuses , rules   |

**Dica:** Para o dashboard, usar `ps=1` com `facets=severities,types` retorna apenas as agregações sem carregar os issues individuais.

### 1.4 GET /api/measures/search\_history

Histórico temporal de métricas para gráficos de tendência.

| Parâmetro | Obrigatório | Descrição                               |
|-----------|-------------|-----------------------------------------|
| component | Sim         | Chave do projeto                        |
| metrics   | Sim         | Lista de métricas separadas por vírgula |
| from      | Não         | Data inicial ( yyyy-MM-dd )             |

|    |     |            |
|----|-----|------------|
| to | Não | Data final |
|----|-----|------------|

## 2. Métricas Relevantes para o Autosserviço

Uma única chamada a `/api/measures/component` pode trazer todas as métricas abaixo:

### 2.1 Subconjunto mínimo recomendado

| Métrica                 | Chave API                       | Tipo     | Descrição                     |
|-------------------------|---------------------------------|----------|-------------------------------|
| Quality Gate            | <code>alert_status</code>       | OK/ERROR | Indicador principal de saúde  |
| Cobertura               | <code>coverage</code>           | %        | Cobertura combinada de testes |
| Bugs                    | <code>bugs</code>               | Contagem | Total de bugs detectados      |
| Vulnerabilidades        | <code>vulnerabilities</code>    | Contagem | Total de vulnerabilidades     |
| Code Smells             | <code>code_smells</code>        | Contagem | Problemas de manutenibilidade |
| Rating Confiabilidade   | <code>reliability_rating</code> | A–E      | Baseado nos bugs              |
| Rating Segurança        | <code>security_rating</code>    | A–E      | Baseado nas vulnerabilidades  |
| Rating Manutenibilidade | <code>sqale_rating</code>       | A–E      | Baseado nos code smells       |

### 2.2 Métricas complementares (exibição opcional)

| Métrica                    | Chave API                                | Tipo     |
|----------------------------|------------------------------------------|----------|
| Dívida técnica             | <code>sqale_index</code>                 | Minutos  |
| Duplicações                | <code>duplicated_lines_density</code>    | %        |
| Linhas de código           | <code>ncloc</code>                       | Contagem |
| Distribuição por linguagem | <code>ncloc_language_distribution</code> | Texto    |
| Complexidade cognitiva     | <code>cognitive_complexity</code>        | Contagem |

### 2.3 Métricas de código novo (New Code Period)

| Métrica                         | Chave API                        |
|---------------------------------|----------------------------------|
| Cobertura no código novo        | <code>new_coverage</code>        |
| Bugs no código novo             | <code>new_bugs</code>            |
| Vulnerabilidades no código novo | <code>new_vulnerabilities</code> |
| Code smells no código novo      | <code>new_code_smells</code>     |

## 3. Autenticação

### 3.1 Tipo de token necessário

| Tipo                   | Escopo                                | Uso                                                              |
|------------------------|---------------------------------------|------------------------------------------------------------------|
| User Token             | Permissões do usuário emissor         | <b>Recomendado</b> — consulta de métricas, issues, quality gates |
| Project Analysis Token | Execução de scan em um projeto        | Apenas para CI/CD                                                |
| Global Analysis Token  | Execução de scan em todos os projetos | Apenas para CI/CD                                                |

O **User Token** é o único que permite consultas de leitura na API. Deve ser gerado em **User > My Account > Security** na UI do SonarQube.

### 3.2 Como autenticar

**Método recomendado — Bearer Token:**

```
Authorization: Bearer squ_abc123...
```

**Alternativa — Basic Auth (token como username, senha vazia):**

```
Authorization: Basic base64(squ_abc123...:)
```

### 3.3 Variáveis de ambiente necessárias

| Variável        | Tipo                  | Descrição                        |
|-----------------|-----------------------|----------------------------------|
| SONARQUBE_URL   | Privada (server-side) | URL da instância SonarQube       |
| SONARQUBE_TOKEN | Privada (server-side) | User Token com acesso de leitura |

## 4. Estrutura de Dados Proposta para o Frontend

```
interface SonarQubeMetrics {
  projectKey: string
  projectName: string
  qualityGate: 'OK' | 'ERROR'
  coverage: number
  bugs: number
  vulnerabilities: number
  codeSmells: number
  technicalDebt: number // em minutos
  duplications: number // percentual
  reliabilityRating: 'A' | 'B' | 'C' | 'D' | 'E'
  securityRating: 'A' | 'B' | 'C' | 'D' | 'E'
  maintainabilityRating: 'A' | 'B' | 'C' | 'D' | 'E'
  linesOfCode: number
  lastAnalysis: string // ISO 8601 date
}
```

### Mapeamento API → Interface

| Campo da interface    | Chave da API                        | Transformação                                    |
|-----------------------|-------------------------------------|--------------------------------------------------|
| qualityGate           | alert_status                        | Direto ( "OK" / "ERROR" )                        |
| coverage              | coverage                            | parseFloat(value)                                |
| bugs                  | bugs                                | parseInt(value)                                  |
| vulnerabilities       | vulnerabilities                     | parseInt(value)                                  |
| codeSmells            | code_smells                         | parseInt(value)                                  |
| technicalDebt         | sqale_index                         | parseInt(value)                                  |
| duplications          | duplicated_lines_density            | parseFloat(value)                                |
| reliabilityRating     | reliability_rating                  | { 1:'A', 2:'B', 3:'C', 4:'D', 5:'E' }<br>[value] |
| securityRating        | security_rating                     | Idem                                             |
| maintainabilityRating | sqale_rating                        | Idem                                             |
| linesOfCode           | ncloc                               | parseInt(value)                                  |
| lastAnalysis          | via<br>/api/project_analyses/search | Data da análise mais recente                     |

## 5. Arquitetura de Integração

Seguindo o padrão de 5 camadas já estabelecido no Autosserviço:

```
Dashboard Page (src/app/dashboard/page.tsx)
|
SonarQubeCard (src/components/dashboard/SonarQubeCard/index.tsx)
|
useSonarQubeMetrics (src/hooks/useSonarQubeMetrics.ts)
|
API Route (src/app/api/sonarqube/metrics/route.ts)
|
Server Action (src/actions/sonarqube.ts) --> SonarQube API
```

## Arquivos a serem criados

| Arquivo                                                       | Camada        | Finalidade                                                 |
|---------------------------------------------------------------|---------------|------------------------------------------------------------|
| <code>src/types/sonarqube.ts</code>                           | Tipos         | Interface <code>SonarQubeMetrics</code> e tipos auxiliares |
| <code>src/lib/sonarqube.server.ts</code>                      | Biblioteca    | Cliente HTTP com Bearer token (server-only)                |
| <code>src/actions/sonarqube.ts</code>                         | Server Action | Lógica de busca, parse e transformação de métricas         |
| <code>src/app/api/sonarqube/metrics/route.ts</code>           | API Route     | Endpoint autenticado (validação de sessão + proxy)         |
| <code>src/hooks/useSonarQubeMetrics.ts</code>                 | Hook          | React Query wrapper com cache e refetch                    |
| <code>src/components/dashboard/SonarQubeCard/index.tsx</code> | Componente    | Card de visualização no dashboard                          |

## Arquivos a serem modificados

| Arquivo                                 | Alteração                                                           |
|-----------------------------------------|---------------------------------------------------------------------|
| <code>src/app/dashboard/page.tsx</code> | Importar e montar o <code>SonarQubeCard</code> no grid              |
| <code>.env.example</code>               | Adicionar <code>SONARQUBE_URL</code> e <code>SONARQUBE_TOKEN</code> |

## 6. Considerações Técnicas

### 6.1 Paginação e limites

| Aspecto                    | Valor                                                          |
|----------------------------|----------------------------------------------------------------|
| Máximo por página          | 500 itens                                                      |
| Limite total de resultados | 10.000 itens                                                   |
| Paginação                  | Parâmetros <code>p</code> (página) e <code>ps</code> (tamanho) |

Para o dashboard, a paginação não será um problema: as chamadas de métricas retornam um único objeto por projeto.

### 6.2 Rate limiting

- **Self-hosted (provável cenário da SME):** Sem rate limiting nativo
- **SonarQube Cloud:** Possui rate limiting (HTTP 429)

### 6.3 Cache e frequência de atualização

As análises do SonarQube são executadas no CI/CD (não em tempo real). Recomendação:

- **Polling via React Query:** `refetchInterval` de 5 a 15 minutos
- **staleTime:** 5 minutos (evita chamadas redundantes entre navegações)
- Consistente com a abordagem já utilizada nas integrações Zabbix e Azure DevOps

### 6.4 Segurança

- Token permanece exclusivamente server-side (API Route como proxy)
- Mesma abordagem das integrações existentes (Zabbix, Azure DevOps)
- Sem exposição de credenciais ao navegador

### 6.5 Validação da captura da Bruna

A task menciona validar se a “captura já iniciada por Bruna pode ser aproveitada como base”. Pendente verificação com a equipe sobre:

- Qual ferramenta/script foi utilizado
- Quais métricas foram capturadas
- Se há token de acesso já provisionado

## 7. Conclusão e Estimativa de Esforço

### 7.1 Viabilidade

| Aspecto                       | Resultado                                                                    |
|-------------------------------|------------------------------------------------------------------------------|
| Consumo de métricas via API   | <b>Viável</b> — API REST documentada, endpoints cobrem todas as necessidades |
| Autenticação                  | <b>Viável</b> — Bearer token, mesma abordagem das integrações existentes     |
| Integração com o Autosserviço | <b>Viável</b> — Segue o padrão de 5 camadas sem alteração arquitetural       |
| Métricas disponíveis          | <b>Viável</b> — Todas as métricas solicitadas são acessíveis                 |
| Múltiplos projetos            | <b>Viável</b> — Uma chamada por projeto, paralelizável                       |

### 7.2 O que é possível



- Exibir métricas consolidadas por projeto (cobertura, bugs, vulnerabilidades, code smells, ratings)
- Mostrar status do Quality Gate como semáforo visual
- Apresentar distribuição de issues por severidade e tipo
- Montar gráficos de tendência temporal
- Consultar dados de múltiplos projetos em paralelo

### 7.3 O que não é possível

- Acionar análises via API (requer integração com o CI/CD)
- Métricas em tempo real (dependem da execução do scanner no pipeline)
- Acessar projetos sem permissão no token do usuário

### 7.4 Estimativa de esforço

| Etapa                           | Esforço       |
|---------------------------------|---------------|
| Tipos + Lib server + Action     | 4h            |
| API Route + Hook                | 4h            |
| Componente (card no dashboard)  | 8h            |
| Testes (unitários + integração) | 4–8h          |
| <b>Total estimado</b>           | <b>20–24h</b> |

### 7.5 Pendências para início da implementação

1. URL da instância SonarQube da SME
2. User Token com acesso de leitura aos projetos
3. Lista de project keys dos projetos a serem monitorados
4. Validação do trabalho da Bruna (captura de métricas existente)

**Importante:** Sem as pendências acima resolvidas, não é possível iniciar a implementação. A viabilidade técnica está confirmada; a depender apenas de acesso à infraestrutura.